

BEC305 Python Programming — Module 4 Quiz

Classes and Objects

Field	Value
Subject code	BEC305
Module	4 — Classes and Objects
Duration	45 minutes
Maximum marks	50 (Part A 20 + Part B 30)
Attempts allowed	1
Passing marks	20
Textbook	Al Sweigart, Automate the Boring Stuff with Python, 1st ed., 2015

Instructions

- Part A carries 20 one-mark multiple choice questions. Exactly one option is correct.
- Part B carries 13 short questions totalling 30 marks.
- State clearly whether a function you write is pure or a modifier.
- Where a question asks what is printed, give the exact output.
- No computers or interpreters are permitted during the quiz.

Course outcome and Bloom coverage

Band	Questions	Share	Target
L1–L2 Remember / Understand	8	24%	~30%
L3–L4 Apply / Analyse	16	48%	~45%
L5–L6 Evaluate / Create	9	27%	~25%

Course outcome spread: CO4: 33. Total questions: 33.

Part A — Multiple choice (20 × 1 mark = 20 marks)

A1. What does a class statement create? [C04 | L1 | 1 mark | easy | TB2 Ch. 15]

- (A) A new type
- (B) A new instance
- (C) A new module
- (D) A new function

A2. What happens when you assign to a misspelled attribute, as in `blank.Y = 5` when `y` was intended?

[C04 | L3 | 1 mark | medium | TB2 Ch. 15]

- (A) A new attribute `Y` is created silently and `y` keeps its old value
- (B) `AttributeError` is raised

- (C) y is updated anyway, since names are case-insensitive
- (D) TypeError is raised

A3. When does reading an attribute raise AttributeError? [C04 | L2 | 1 mark | easy | TB2 Ch. 15]

- (A) When the attribute was never assigned on that instance or its class
- (B) Whenever the attribute is None
- (C) Whenever the class has no __init__
- (D) Only for built-in types

A4. What does box.corner.x express when box is a Rectangle holding a Point? [C04 | L2 | 1 mark | medium | TB2 Ch. 15]

- (A) The x attribute of the Point held by the Rectangle
- (B) A method call on box
- (C) A dictionary lookup
- (D) An attribute named corner.x on box

A5. If a function modifies an attribute of an object passed to it, what does the caller see? [C04 | L4 | 1 mark | medium | TB2 Ch. 15]

- (A) The change, because the function received a reference rather than a copy
- (B) Nothing, because arguments are copied
- (C) A TypeError
- (D) The change only if the function returns the object

A6. After b = copy.copy(box) where box holds a Point corner, what is b.corner is box.corner? [C04 | L4 | 1 mark | hard | TB2 Ch. 15]

- (A) True
- (B) False
- (C) None
- (D) It raises an exception

A7. What is the defining property of a pure function? [C04 | L2 | 1 mark | easy | TB2 Ch. 16]

- (A) It modifies nothing outside itself and returns a new value
- (B) It returns None
- (C) It takes no arguments
- (D) It is defined inside a class

A8. What does a modifier conventionally return? [C04 | L2 | 1 mark | medium | TB2 Ch. 16]

- (A) None
- (B) The modified object
- (C) A copy of the object
- (D) A Boolean indicating success

A9. Why is converting a Time to seconds since midnight before arithmetic a good idea? [C04 | L4 | 1 mark | hard | TB2 Ch. 16]

- (A) The invariant is restored by the conversion back, so no carrying logic is needed
- (B) It is faster than attribute arithmetic
- (C) It uses less memory
- (D) It is required by the language

A10. What distinguishes prototype-and-patch from designed development? [C04 | L5 | 1 mark | hard | TB2 Ch. 16]

- (A) Patching accumulates special cases; design finds a representation in which they do not arise
- (B) Patching is always wrong
- (C) Design produces a first working version faster
- (D) They produce identical code

A11. In a method definition, what is self? [C04 | L2 | 1 mark | easy | TB2 Ch. 17]

- (A) The instance the method was called on, passed automatically
- (B) A reserved keyword
- (C) The class itself
- (D) A copy of the instance

A12. Calling t.print_time() on a method defined as def print_time(): raises what? [C04 | L3 | 1 mark | medium | TB2 Ch. 17]

- (A) TypeError, because the method takes 0 positional arguments but 1 was given
- (B) AttributeError
- (C) SyntaxError
- (D) Nothing; it works

A13. When is __init__ called? [C04 | L2 | 1 mark | easy | TB2 Ch. 17]

- (A) Automatically, when the class is called to create an instance
- (B) Only when you call it explicitly
- (C) Once per class definition
- (D) When the object is garbage collected

A14. Why is a mutable default such as def __init__(self, items=[]) a bug? [C04 | L4 | 1 mark | hard | TB2 Ch. 17]

- (A) The default is evaluated once at definition time and shared by every instance
- (B) Python raises SyntaxError
- (C) It makes the class unhashable
- (D) Defaults cannot be lists

A15. What does print(t) show when the class defines no __str__? [C04 | L2 | 1 mark | easy | TB2 Ch. 17]

- (A) The class name and a memory address
- (B) An empty string
- (C) The attribute values
- (D) None

A16. What must __str__ do with its result? [C04 | L3 | 1 mark | medium | TB2 Ch. 17]

- (A) Return it as a string
- (B) Print it
- (C) Store it on the instance
- (D) Yield it

A17. Which special method does Python call for the expression t1 + t2? [C04 | L3 | 1 mark | medium | TB2 Ch. 17]

- (A) `t1.__add__(t2)`
- (B) `t2.__add__(t1)`
- (C) `__sum__`
- (D) `t1.add(t2)`

A18. What is the consequence of defining `__eq__` without `__hash__`? [C04 | L5 | 1 mark | hard | TB2 Ch. 17]

- (A) Instances become unhashable and cannot be used as dictionary keys
- (B) Equality stops working
- (C) The class cannot be instantiated
- (D) Nothing changes

A19. What does a long chain of `isinstance` branches usually indicate? [C04 | L5 | 1 mark | hard | TB2 Ch. 17-18]

- (A) That a polymorphic interface would serve better
- (B) That the code is well designed
- (C) That the class needs more attributes
- (D) That `__init__` is missing

A20. When is a full class the right choice over a `namedtuple` or `dataclass`? [C04 | L5 | 1 mark | medium | TB2 Ch. 15-18]

- (A) When behaviour attaches to the data or an invariant must be enforced on construction
- (B) Whenever two or more values travel together
- (C) Whenever the values are numbers
- (D) Always, since classes are the Pythonic form

Part B — Descriptive (13 questions, 30 marks)

B1. Explain what a class gives you that a dictionary does not, and when the extra ceremony is justified.

[C04 | L5 | 3 marks | TB2 Ch. 15]

B2. Explain the difference between a pure function and a modifier, and how each is tested. [C04 | L4 | 3 marks | TB2 Ch. 16]

B3. Explain why converting a `Time` to seconds since midnight before arithmetic removes the need for carrying logic. [C04 | L5 | 3 marks | TB2 Ch. 16]

B4. Explain what `__init__` guarantees and give two ways of getting it wrong. [C04 | L4 | 3 marks | TB2 Ch. 17]

B5. Why does assigning to a misspelled attribute not raise an error? [C04 | L3 | 2 marks | TB2 Ch. 15]

B6. Explain why a shallow copy of a `Rectangle` holding a `Point` is not independent, and how to check. [C04 | L4 | 2 marks | TB2 Ch. 15]

B7. Explain what `self` is and why a method definition appears to have one more parameter than its call. [C04 | L3 | 2 marks | TB2 Ch. 17]

B8. Explain the purpose of `__str__` and why `__repr__` matters too. [C04 | L4 | 2 marks | TB2 Ch. 17]

B9. Explain how operator overloading works and give one rule for when to use it. [C04 | L4 | 2 marks | TB2 Ch. 17]

B10. What breaks when you define `__eq__` without `__hash__`, and why? [C04 | L5 | 2 marks | TB2 Ch. 17]

B11. Distinguish type-based dispatch from polymorphism, and say when each is appropriate. [C04 | L5 | 2 marks | TB2 Ch. 17-18]

B12. Explain why a function can change an object passed to it, and what discipline that requires. [C04 | L4 | 2 marks | TB2 Ch. 15-16]

B13. A program needs a 2-D point. Give the structure you would choose under two different requirement sets. [C04 | L5 | 2 marks | TB2 Ch. 15-18]

Answer key — Part A

Q	Answer	CO	Bloom	Reference
A1	A	CO4	L1	TB2 Ch. 15
A2	A	CO4	L3	TB2 Ch. 15
A3	A	CO4	L2	TB2 Ch. 15
A4	A	CO4	L2	TB2 Ch. 15
A5	A	CO4	L4	TB2 Ch. 15
A6	A	CO4	L4	TB2 Ch. 15
A7	A	CO4	L2	TB2 Ch. 16
A8	A	CO4	L2	TB2 Ch. 16
A9	A	CO4	L4	TB2 Ch. 16
A10	A	CO4	L5	TB2 Ch. 16
A11	A	CO4	L2	TB2 Ch. 17
A12	A	CO4	L3	TB2 Ch. 17
A13	A	CO4	L2	TB2 Ch. 17
A14	A	CO4	L4	TB2 Ch. 17
A15	A	CO4	L2	TB2 Ch. 17
A16	A	CO4	L3	TB2 Ch. 17
A17	A	CO4	L3	TB2 Ch. 17
A18	A	CO4	L5	TB2 Ch. 17
A19	A	CO4	L5	TB2 Ch. 17-18
A20	A	CO4	L5	TB2 Ch. 15-18

Part A — why each option is right or wrong

A1. Correct answer: (A) A new type

- (A) Correct - the class defines the type; calling it creates instances.
- (B) Wrong - an instance is created by calling the class, not by defining it.
- (C) Wrong - a module is a file; a class lives inside one.
- (D) Wrong - def creates a function; class creates a type.

A2. Correct answer: (A) A new attribute Y is created silently and y keeps its old value

- (A) Correct - assignment creates attributes, so a typo makes a second one and nothing reports it.
- (B) Wrong - AttributeError is raised on reading a missing attribute, never on writing one.
- (C) Wrong - Python names are case sensitive throughout.
- (D) Wrong - no type rule is violated.

A3. Correct answer: (A) When the attribute was never assigned on that instance or its class

- (A) Correct - the read fails because the name is absent, which is usually a misspelled write earlier.
- (B) Wrong - an attribute holding None is present and reads fine.
- (C) Wrong - a class without __init__ is legal; attributes can be assigned later.
- (D) Wrong - it applies to any object.

A4. Correct answer: (A) The x attribute of the Point held by the Rectangle

- (A) Correct - each dot is one hop, resolved left to right.
- (B) Wrong - a call would need parentheses.
- (C) Wrong - dot notation reaches attributes, not dictionary keys.
- (D) Wrong - the dot separates two attribute lookups; it is not part of a single name.

A5. Correct answer: (A) The change, because the function received a reference rather than a copy

- (A) Correct - Python never copies on a call, so mutation is visible to the caller.
- (B) Wrong - arguments are never copied; this is the misconception the unit exists to correct.
- (C) Wrong - no type error arises from mutating an attribute.
- (D) Wrong - the return value is irrelevant; the object itself was changed.

A6. Correct answer: (A) True

- (A) Correct - a shallow copy duplicates the outer object only; the embedded Point is shared.
- (B) Wrong - that would require copy.deepcopy.
- (C) Wrong - is always returns a Boolean.
- (D) Wrong - the comparison is perfectly legal.

A7. Correct answer: (A) It modifies nothing outside itself and returns a new value

- (A) Correct - its entire effect is its return value, which is what makes it testable in one line.
- (B) Wrong - returning None is the convention for a modifier, the opposite case.
- (C) Wrong - a pure function usually takes arguments; it simply does not modify them.
- (D) Wrong - purity has nothing to do with where the function is defined.

A8. Correct answer: (A) None

- (A) Correct - the None signals that the whole effect was the in-place change.
- (B) Wrong - returning the object makes it impossible to tell a modifier from a pure function at the call site.
- (C) Wrong - a modifier makes no copy; that is the point of it.
- (D) Wrong - success is signalled by not raising.

A9. Correct answer: (A) The invariant is restored by the conversion back, so no carrying logic is needed

- (A) Correct - divmod handles every carry, so the special cases disappear rather than being patched.
- (B) Wrong - speed is not the motivation; correctness is.
- (C) Wrong - memory is unchanged.
- (D) Wrong - nothing in the language requires it; it is a design choice.

A10. Correct answer: (A) Patching accumulates special cases; design finds a representation in which they do not arise

- (A) Correct - the canonical representation dissolves the special cases rather than handling them.
- (B) Wrong - prototyping is the right approach while the problem is still unfamiliar.
- (C) Wrong - prototyping reaches a first working version faster; design is cheaper to extend.
- (D) Wrong - the patched version carries branches the designed one does not need.

A11. Correct answer: (A) The instance the method was called on, passed automatically

- (A) Correct - the instance before the dot becomes the first argument.
- (B) Wrong - self is a convention, not a keyword; any name would work, though none should be used.
- (C) Wrong - the class is what cls refers to in a classmethod.
- (D) Wrong - no copy is made; self is the instance itself.

A12. Correct answer: (A) TypeError, because the method takes 0 positional arguments but 1 was given

- (A) Correct - the instance is always passed, so the definition must have a parameter to receive it.
- (B) Wrong - the method exists, so the lookup succeeds; the call is what fails.
- (C) Wrong - the definition parses fine.
- (D) Wrong - the automatic self argument has nowhere to go.

A13. Correct answer: (A) Automatically, when the class is called to create an instance

- (A) Correct - which is what guarantees no code path yields a half-built instance.
- (B) Wrong - Python invokes it for you on instantiation.
- (C) Wrong - it runs once per instance, not once per class.
- (D) Wrong - that is `__del__`.

A14. Correct answer: (A) The default is evaluated once at definition time and shared by every instance

- (A) Correct - one list is created at definition and every instance that omits the argument shares it.
- (B) Wrong - it is valid syntax, which is why the bug is common.
- (C) Wrong - hashability is unaffected.
- (D) Wrong - a list default is permitted; it simply behaves surprisingly.

A15. Correct answer: (A) The class name and a memory address

- (A) Correct - the default representation tells a reader nothing about the value.
- (B) Wrong - a default representation is always produced.
- (C) Wrong - attribute values appear only if you write a method that shows them.
- (D) Wrong - `print` never shows `None` for an object.

A16. Correct answer: (A) Return it as a string

- (A) Correct - the caller does the printing; `__str__` only supplies the text.
- (B) Wrong - printing inside it makes the output appear twice and `print(t)` also show `None`.
- (C) Wrong - no storage is involved.
- (D) Wrong - it is an ordinary method, not a generator.

A17. Correct answer: (A) `t1.__add__(t2)`

- (A) Correct - the left operand is the receiver.
- (B) Wrong - that reflected form, `__radd__`, is tried only if the first returns `NotImplemented`.
- (C) Wrong - there is no `__sum__` special method.
- (D) Wrong - the operator maps to the dunder name, not to a plain method.

A18. Correct answer: (A) Instances become unhashable and cannot be used as dictionary keys

- (A) Correct - Python sets `__hash__` to `None` when `__eq__` is defined, unless you supply one.
- (B) Wrong - equality works; it is hashing that breaks.
- (C) Wrong - instantiation is unaffected.
- (D) Wrong - the silent loss of hashability is a real and common surprise.

A19. Correct answer: (A) That a polymorphic interface would serve better

- (A) Correct - each new type otherwise means another branch, where polymorphism needs none.
- (B) Wrong - unbounded branch growth is the classic smell this unit names.
- (C) Wrong - attribute count is unrelated.
- (D) Wrong - dispatch style has nothing to do with initialisation.

A20. Correct answer: (A) When behaviour attaches to the data or an invariant must be enforced on construction

- (A) Correct - ceremony is justified by behaviour or by an invariant, not by grouping alone.
- (B) Wrong - a tuple or namedtuple carries the same information for far less code.
- (C) Wrong - the type of the values is irrelevant to the choice.
- (D) Wrong - a class containing only `__init__` reproduces by hand what a dataclass gives free.

Part B — model answers and marking scheme

B1 (3 marks) — CO4, L5, TB2 Ch. 15

Question. Explain what a class gives you that a dictionary does not, and when the extra ceremony is justified.

Model answer. A class names a type, so a `Point` with `x` and `y` reads as a point everywhere it appears, where a dictionary with two keys reads as whatever the reader guesses. It also gives one place to attach the operations belonging to the data, and, through `__init__`, a guarantee that no instance exists in a half-built state. The ceremony is justified once behaviour attaches to the data or an invariant must be enforced; while nothing but attribute access happens, a tuple, namedtuple or dictionary carries the same information for a fraction of the code.

Point required	Marks
Correct statement of naming and behaviour attachment	1
Correct criterion for when a class is justified	2
Total	3

B2 (3 marks) — CO4, L4, TB2 Ch. 16

Question. Explain the difference between a pure function and a modifier, and how each is tested.

Model answer. A pure function reads its arguments, changes nothing outside itself and returns a new value, so calling it twice with the same arguments gives the same result and the arguments survive

unchanged. A modifier changes an object it was given and conventionally returns None, so its effect is invisible at the call site. A pure function is tested with a single comparison of the return value against an expected one; a modifier must be tested by constructing an object, calling the function, and then inspecting the object afterwards. The modifier also makes call order significant, which the pure form does not.

Point required	Marks
Correct definition of both	1
Correct testing approach for each	1
Correct note about call ordering or call-site invisibility	1
Total	3

B3 (3 marks) — CO4, L5, TB2 Ch. 16

Question. Explain why converting a Time to seconds since midnight before arithmetic removes the need for carrying logic.

Model answer. A Time has an invariant: minute and second must lie between 0 and 59. Adding attribute by attribute breaks it, so the prototype approach patches each overflow with a conditional that subtracts 60 and carries. Converting to a single canonical number with `time_to_int`, doing the arithmetic there, and converting back with `int_to_time` restores the invariant by construction, because `divmod` performs every carry. Three conditional patches become zero, and the special cases dissolve rather than being handled.

Point required	Marks
Correct statement of the invariant	1
Correct description of the canonical-form round trip	1
Correct conclusion that the special cases disappear	1
Total	3

B4 (3 marks) — CO4, L4, TB2 Ch. 17

Question. Explain what `__init__` guarantees and give two ways of getting it wrong.

Model answer. `__init__` runs automatically when the class is called, so it is the one place that can guarantee every instance carries every attribute the class requires and satisfies any invariant before anyone else touches it. Assigning attributes by hand after creation leaves a window in which the object is incomplete, and a later read raises `AttributeError` far from the cause. Two ways of getting it wrong are giving a parameter a mutable default such as `items=[]`, which is evaluated once at definition and shared by every instance, and assigning a required attribute somewhere other than `__init__`, which reopens the incomplete window.

Point required	Marks
Correct statement of the guarantee	1
Mutable default explained correctly	1
Second failure mode described	1
Total	3

B5 (2 marks) — CO4, L3, TB2 Ch. 15

Question. Why does assigning to a misspelled attribute not raise an error?

Model answer. Attributes are created by assignment rather than declared, and they live in a per-instance dictionary, so `blank.Y = 5` simply adds a new entry named `Y`. The intended attribute `y` keeps its old value and nothing reports the divergence. The only signal arrives later, when code reads an attribute that was never assigned and raises `AttributeError`, typically far from the line that caused it. Setting every attribute in `__init__` narrows the window in which this can happen.

Point required	Marks
Correct explanation of assignment-creates-attribute	1
Correct account of the delayed <code>AttributeError</code>	1
Total	2

B6 (2 marks) — CO4, L4, TB2 Ch. 15

Question. Explain why a shallow copy of a `Rectangle` holding a `Point` is not independent, and how to check.

Model answer. `copy.copy` builds a new `Rectangle` and copies the attribute references into it, so the new object has its own `width` and `height` slots but its `corner` attribute holds the same `Point` reference as the original. Changing `b.corner.x` therefore changes `box.corner.x` too. Checking `b is box` returns `False` and looks reassuring, but the real test is `b.corner is box.corner`, which returns `True` and reveals the sharing. `copy.deepcopy` rebuilds the embedded `Point` as well and makes both tests `False`.

Point required	Marks
Correct explanation of the shared embedded object	1
Correct diagnostic using <code>is</code> on the embedded attribute	1
Total	2

B7 (2 marks) — CO4, L3, TB2 Ch. 17

Question. Explain what `self` is and why a method definition appears to have one more parameter than its call.

Model answer. When a method is called as `instance.method(args)`, Python passes the instance before the dot as the first positional argument, conventionally named `self`. The definition must therefore declare a parameter to receive it, so `def print_time(self)` is called as `t.print_time()` with no visible argument. Omitting `self` gives `TypeError` saying the method takes 0 positional arguments but 1 was given. `self` is a convention rather than a keyword, but no other name should be used.

Point required	Marks
Correct description of the automatic first argument	1
Correct error named when <code>self</code> is omitted	1
Total	2

B8 (2 marks) — CO4, L4, TB2 Ch. 17

Question. Explain the purpose of `__str__` and why `__repr__` matters too.

Model answer. `__str__` returns the text Python shows when an instance is printed or passed to `str()`; without it, printing shows the class name and a memory address, which helps nobody debugging. It must return a string rather than printing one, or the output appears twice and print also shows `None`. `__repr__` is the developer-facing counterpart and is what the interactive shell and container displays use, so a list of objects shows addresses even when `__str__` is defined. If only one is written, `__repr__` is the more useful.

Point required	Marks
Correct purpose of <code>__str__</code> with the return-not-print rule	1
Correct role of <code>__repr__</code> and the container case	1
Total	2

B9 (2 marks) — CO4, L4, TB2 Ch. 17

Question. Explain how operator overloading works and give one rule for when to use it.

Model answer. Python translates an operator expression into a special-method call on the left operand, so `t1 + t2` becomes `t1.__add__(t2)`, and the same mechanism covers `__sub__`, `__mul__`, `__eq__` and the rest. Returning `NotImplemented` rather than raising lets Python try the reflected operation on the other operand, which is how mixed-type arithmetic works. The rule is to overload only when the operator carries its conventional meaning for the type; giving `+` a surprising meaning is worse than a plainly named method, because the reader has no way to suspect it.

Point required	Marks
Correct translation mechanism with an example	1
Correct usage rule, ideally with the <code>NotImplemented</code> point	1
Total	2

B10 (2 marks) — CO4, L5, TB2 Ch. 17

Question. What breaks when you define `__eq__` without `__hash__`, and why?

Model answer. Python sets `__hash__` to `None` on any class that defines `__eq__` unless a `__hash__` is supplied, so instances become unhashable and can no longer be used as dictionary keys or placed in a set. The reason is consistency: two objects that compare equal must hash equally, and Python cannot derive the new hash from the new equality automatically. The remedies are to define `__hash__` explicitly in terms of the same attributes `__eq__` uses, or to use a frozen dataclass, which generates both together.

Point required	Marks
Correct statement of the unhashability consequence	1
Correct reason and at least one remedy	1
Total	2

B11 (2 marks) — CO4, L5, TB2 Ch. 17-18

Question. Distinguish type-based dispatch from polymorphism, and say when each is appropriate.

Model answer. Type-based dispatch inspects an operand with `isinstance` and branches, so one method can handle several argument types explicitly. Polymorphism writes the function against an interface instead, so any object supplying the required methods works, including types written later. Python favours

the polymorphic form through duck typing, where suitability is having the method rather than being the class. Dispatch is appropriate when the behaviours genuinely differ in kind and the set of types is small and closed; a growing chain of three or more instance branches is the signal that polymorphism was the right answer.

Point required	Marks
Correct definition of both	1
Correct criterion for choosing, including the branch-count heuristic	1
Total	2

B12 (2 marks) — CO4, L4, TB2 Ch. 15-16

Question. Explain why a function can change an object passed to it, and what discipline that requires.

Model answer. Argument passing in Python binds the parameter to the same object the caller holds; nothing is copied, so assigning to an attribute of that parameter changes the caller's object. Nothing at the call site distinguishes a function that does this from one that does not, so the discipline is entirely conventional: name a modifying function with a verb that announces the change, state in the docstring exactly which attributes are touched, and return None so the absent return value signals the in-place effect. Where the caller must be protected, pass a copy or prefer a pure function that returns a new object.

Point required	Marks
Correct explanation of reference passing	1
Correct naming, documentation and return-None discipline	1
Total	2

B13 (2 marks) — CO4, L5, TB2 Ch. 15-18

Question. A program needs a 2-D point. Give the structure you would choose under two different requirement sets.

Model answer. If the program only ever reads .x and .y and never computes anything, a namedtuple gives named fields with tuple economy in a single line, and a plain tuple would do if even the names are unnecessary. If distance and midpoint operations attach to the data, a dataclass earns its keep: three lines generate __init__, __repr__ and __eq__ and leave room for methods. A full class is warranted only when an invariant must be enforced on construction, such as validating that both coordinates are finite. A class containing nothing but __init__ is evidence a lighter structure was available.

Point required	Marks
Two requirement sets with correctly matched structures	1
Correct justification, including the only-__init__ smell	1
Total	2